

Lab 2: An RPN Calculator

CAS CS 320: Concepts of Programming Languages

September 23, 2026

Throughout this course, you'll build interpreters to practice the concepts we cover. For these interpreters you'll be given a programming language *specification*. As we've said from day one, a specification consists of three things: syntax rules, typing rules, and semantic rules. This is our first interpreter so we'll give a high-level description of these things. As we get further along, the specifications will become more formal and concise.

1 Overview

One of the simplest programming languages is arithmetic. There are several syntaxes for arithmetic; we'll be using *reverse Polish notation* (RPN). In reverse Polish notation, arithmetic operators appear after their arguments. This has the benefit of not requiring parentheses, which makes RPN expressions easier to parse and evaluate. We further require that all numbers and operators that appear in an RPN expression are separated by whitespace. The following table contains translations of arithmetic expressions in standard infix notation and in RPN.

Infix notation	RPN
$(1 + 2) + 3$	1 2 + 3 +
$1 + (2 - 3)$	1 2 3 - +
$1/2 \times (3 + 4)$	1 2 / 3 4 + ×

2 Syntax

A program in our language is given by the following BNF grammar:

$$\begin{aligned}\langle \text{prog} \rangle &= \{ \langle \text{com} \rangle \} \\ \langle \text{com} \rangle &= \langle \text{op} \rangle \mid \langle \text{num} \rangle \\ \langle \text{op} \rangle &= + \mid - \mid * \mid /\end{aligned}$$

where $\langle \text{num} \rangle$ is any nonempty contiguous sequence of digits $[0-9]^+$.

The above BNF grammar says that a program ($\langle \text{prog} \rangle$) is a sequence of commands ($\langle \text{com} \rangle$) and a command is either an operator ($\langle \text{op} \rangle$) or a number

(`<num>`). An operator is either an addition symbol (+), subtraction symbol (-), multiplication symbol (*), or division symbol (/). For reasons that will become clear later, numbers aren't described directly in the grammar, but are instead described externally by a regular expression. Again, for our purposes, a number is a contiguous sequence of digits. Programs in this specification are whitespace agnostic. This means it doesn't matter how many or what kind of spaces there are between each command in a program. The following table contains the previous examples written as programs in our language.

RPN	Program
1 2 + 3 +	1 2 + 3 +
1 2 3 - +	1 2 3 - +
1 2 / 3 4 + ×	1 2 / 3 4 + *

3 Typing

Our language has no type system.¹

4 Semantics

The evaluation of a program in our language is given by the following small-step semantic rules:

$$\begin{array}{c}
\frac{n \text{ is a number}}{\langle \mathcal{S}, n \, p \rangle \longrightarrow \langle n :: \mathcal{S}, p \rangle} \text{ NUM} \\
\\
\frac{}{\langle n :: m :: \mathcal{S}, + \, p \rangle \longrightarrow \langle (m + n) :: \mathcal{S}, p \rangle} \text{ ADD} \\
\\
\frac{}{\langle n :: m :: \mathcal{S}, - \, p \rangle \longrightarrow \langle (m - n) :: \mathcal{S}, p \rangle} \text{ SUB} \\
\\
\frac{}{\langle n :: m :: \mathcal{S}, * \, p \rangle \longrightarrow \langle (m \times n) :: \mathcal{S}, p \rangle} \text{ MUL} \\
\\
\frac{n \neq 0}{\langle n :: m :: \mathcal{S}, / \, p \rangle \longrightarrow \langle \lfloor m/n \rfloor :: \mathcal{S}, p \rangle} \text{ DIV} \qquad \frac{\mathcal{C}_1 \longrightarrow \mathcal{C}_2}{\mathcal{C}_1 \longrightarrow^* \mathcal{C}_2} \text{ REFL} \\
\\
\frac{\mathcal{C}_1 \longrightarrow \mathcal{C}_2 \quad \mathcal{C}_2 \longrightarrow^* \mathcal{C}_3}{\mathcal{C}_1 \longrightarrow^* \mathcal{C}_3} \text{ TRANS}
\end{array}$$

where $\mathcal{S}$ is a stack. The we take the value of a program p to be the integer n such that $\langle \emptyset, p \rangle \longrightarrow^* \langle [n], \epsilon \rangle$ is derivable, where $\emptyset$ is the empty stack and ϵ the empty program. Note that not all programs have values.

¹Another way to say this is that the type system is trivial; every program is typable as a single dummy type $\bullet$.

These semantic rules describe the evaluation of a program in terms of a *configuration* $\langle \mathcal{S}, p \rangle$ which in this case is a program p together with a stack $\mathcal{S}$. Each rule describes what happens to a configuration in a single step of evaluation. For example, the (num) rule says that if the next command in a program is a number, then that number should be pushed onto the stack. The (add) rule says that if the next command is an addition symbol and there are at least two elements on the stack, the top two elements should be added together.

Some rules have *side conditions* (the highlighted premises) which give conditions under which an inference rule can be validly applied. For example, we can't apply the (div) rule if it would require us to divide a number by 0.

5 The Task

You need to implement parsing and evaluation, though you should do evaluation first. For evaluation, you need to implement the function

```
val eval : string list -> int
```

so that `eval p` is the value of the program `p` according to the above specification. The behavior of this function is undefined if `p` does not have a value. Note that we are representing a program as a list of strings. *Hint:* Implement a helper function that takes a stack as an argument.

For parsing, you need to implement the function

```
val split_by_ws : string -> string list
```

which has the same specification as the problem from this week's assignment.